

The BBPay Terminal Interface Redesign

From an app on an Android device to a closed payment terminal operating environment — the case for the redesign, and the answers to the arguments against it

Field	Detail
Document	Corporate report — business case and design rationale
Subject	BBPay / WisePay 2.0 merchant interface, Wiseasy P5 (Android 11)
Status	For review
Version	1.0
Date	24 August 2026
Prepared by	Product & Design, Payment Cards Group
Audience	Executive sponsors, Operations, Merchant Support, Engineering
Author	Yordine Salvador Chimbutane

This report describes the redesigned interface as delivered. Where a capability depends on a privilege we do not yet hold on the fleet, that dependency is stated explicitly in Section 8 rather than assumed away.

Contents

Contents.....	2
1. Executive summary	4
1.1 What this report asks for	4
1.2 At a glance.....	5
2. The problem, as it exists on the counter today	6
2.1 The terminal is a phone that happens to take payments.....	6
2.2 The settings a shop needs are the settings the app does not have.....	6
2.3 Language is the sharpest case.....	6
2.4 Network state is discovered at the worst possible moment	7
2.5 What this costs	7
3. The proposal — BBPay as the terminal's operating environment	8
3.1 The organising principle: one surface, no exits	8
3.2 The status bar — the app's own, not Android's.....	8
3.3 The payment loop	9
3.4 Two tiers, so that the cashier makes no decisions	9
3.5 Pinned, and not by request	10
4. Old against new, in detail.....	11
5. What this is worth	12
5.1 Fewer decisions at the counter, and therefore fewer errors	12
5.2 A bilingual counter that actually works.....	12
5.3 Connectivity becomes a fact, not a discovery	12
5.4 Support cost.....	12
5.5 A defensible boundary	12
5.6 Fleet consistency	13
6. Design decisions worth defending individually	14
6.1 The visual language is the shipped app's, not a new one	14
6.2 Only two languages	14
6.3 The amount and the keypad do not scale with text size.....	14
6.4 Entitlements filter the manager surface	14
6.5 The result screen keeps “unknown”	14
7. Objections, and our answers	15
Objection 1 — Merchants will need a long time to adapt	15
Objection 2 — Touching a certified payment application is a certification risk.....	15
Objection 3 — Locking down Android removes flexibility we will wish we had	15
Objection 4 — Managers will resent being restricted, or will want the full device	16
Objection 5 — Support and field engineering need Android access.....	16
Objection 6 — The status bar is precious space, and a language button does not earn it.....	17
Objection 7 — The lockdown can be defeated, so it is security theatre	17
Objection 8 — Wi-Fi and mobile data cannot be controlled from an ordinary app anyway	17

Objection 9 — Older or less confident staff will struggle with a new interface	18
Objection 10 — Location and attestation features raise privacy and legal questions	18
Objection 11 — This is a large engineering investment in something that already works	19
Objection 12 — Merchants did not ask for this	19
8. Dependencies and open items	20
9. Rollout and how we will know it worked	21
9.1 Phasing	21
9.2 Metrics	21
10. Recommendation	22

1. Executive summary

BBPay today is an application. It runs on a Wiseasy P5, which is a general-purpose Android 11 device, and it shares that device with the Android operating system it happens to sit on. The payment stack inside it is certified, reliable and, on its own terms, finished. The problem is not the payment stack. The problem is everything a merchant has to do that is not a payment.

A cashier who needs a different language, a working Wi-Fi connection, larger text, or simply an answer to the question “is this terminal online right now?” cannot get any of it from the payment app. They have to leave it, enter Android, navigate a settings tree that was designed for a phone, and find their way back. Some of them manage. Many of them call support. A few of them change something they should not have.

The redesign closes that loop. BBPay becomes the terminal's operating environment rather than one app inside somebody else's: a single pinned surface with the app's own status bar, the app's own settings, and a payment flow that is deliberately, almost pedantically, the same flow the merchant already performs today. The settings that a shop genuinely needs are inside the app; the ones a shop does not need are not there at all. The settings that carry risk sit behind the manager's passcode, so that a manager configures the terminal once and their cashiers simply use it.

The central design claim of this report is a narrow one, and we want it stated plainly because the strongest argument against any redesign is retraining cost:

The payment flow is not being changed. Home → amount → method → pay → back to the amount keypad is what a BBPay merchant does today. What changes is everything around that flow — and almost all of it is material the merchant previously had to leave the app to reach.

A cashier who learns nothing new can still take payments on day one. That is the point. The redesign spends its novelty budget on the parts of the product merchants currently fail at, and spends none of it on the part they already do well.

1.1 Purpose

Highlight the necessity of the redesigned interface as the shipping merchant surface for BBPay on the P5 fleet.

1.2 At a glance

Dimension	Today	Redesigned
What the merchant operates	An app on an Android device	A closed terminal environment; Android is not reachable
Change the language	Exit app → Android Settings → System → Languages & input → reorder locales → return	One button in the status bar, visible on every screen, applied instantly
Check or change Wi-Fi	Exit app → Android Settings → Network	Status bar shows state at all times; connection screen is in-app
Change text size	Exit app → Android Settings → Display (and it fights the app's own layout)	Four steps, in-app, keypad and amount deliberately exempt
Who may change what	Whoever holds the terminal	Two tiers: presentation open to cashiers, configuration behind the manager passcode
Configuration decisions asked of a cashier	Unbounded — the whole of Android	None. The manager has already made them
Payment flow	Home → amount → method → pay	Identical, and returns to the keypad rather than home
Certified payment stack	EMV, PIN, keys, gateway, printing	Untouched. The redesign is a presentation layer above the same interfaces

2. The problem, as it exists on the counter today

The following is not a critique of the shipped app's engineering. It is a description of what the product asks of the person holding it, and each item below was read out of the shipped source rather than inferred.

2.1 The terminal is a phone that happens to take payments

The P5 boots Android 11. BBPay is a launcher-registered activity on it. The shipped app has a kiosk mode, and that mode does useful work — it blocks the back button, removes the toolbar's up-arrow, suppresses login dialogs, auto-dismisses print failures. But kiosk mode as shipped does not make the app Device Owner, does not pin the task, and does not disable the notification shade. The device underneath remains a general-purpose computer, and it remains reachable.

The practical consequence is that the terminal's behaviour is not a property of our product. It is a property of whatever state the merchant, their staff, or a previous engineer left the Android installation in.

2.2 The settings a shop needs are the settings the app does not have

The shipped settings screen builds five sections: General, Account, Octopus, Transaction and From server. It is a competent screen. It is also, from a shopkeeper's point of view, a screen about the acquiring relationship rather than about running a terminal.

What a shop actually asks for	Where it lives today
Change the language for a bilingual counter	Not in the app at all. Android system locale
Join a different Wi-Fi network	Not in the app at all. Android Settings
Turn mobile data on or off	Not in the app at all. Android Settings
Make the text bigger for a staff member who cannot read it	Not in the app at all. Android Settings → Display
See whether the terminal is online before a customer is waiting	Partially — a status row on the home screen
Printer copies, auto-print, receipt behaviour	In the app (General)
Change password, device ID, entitlements	In the app (Account, Transaction, From server)

Four of the seven most common merchant requests are answered by sending the merchant out of our product. That is the gap the redesign was commissioned to close.

2.3 Language is the sharpest case

The app is fully bilingual. The English string table carries 788 strings and the Traditional Chinese table carries 772 of them, translated by the team that ships to these merchants. The translation

work is done and it is good. It is simply not reachable: localisation follows the Android system locale, so switching language means leaving the payment app, entering Android's Languages & input screen, reordering the locale list, and coming back.

On a counter with one Cantonese-first and one English-first staff member sharing a shift, that is not a settings change. It is a reason to give up and use the terminal in a language you read slowly. Slow reading at a payment terminal is not a cosmetic problem: it is the mechanism by which a cashier confirms an amount they did not fully parse.

2.4 Network state is discovered at the worst possible moment

A terminal that has silently dropped off Wi-Fi looks exactly like a terminal that is working, right up until a customer has their card out. The merchant then has to diagnose connectivity — a task the app cannot help with, because connectivity is not in the app — while a queue forms.

2.5 What this costs

- Support calls that are, in substance, Android tutoring rather than payment support. Each one is our cost and the merchant's downtime.
- Terminals left in a wrong or degraded state — an unwanted locale, a display timeout that blanks the screen mid-sale, an accessibility setting that breaks the keypad layout — because someone went looking in Android for something and changed a neighbouring option.
- Abandoned or delayed transactions when connectivity fails invisibly.
- An open surface with no meaningful boundary between “the payment product” and “the device”, which makes any assurance about how the terminal is used difficult to give and impossible to enforce.
- A brand experience that ends at the edge of our own activity. The moment a merchant leaves BBPay to do something BBPay made them leave to do, they are using Google's product to fix ours.

3. The proposal — BBPay as the terminal's operating environment

The redesign is native Android, built as a module inside the shipped application. It is not a second app and there is nothing separate to install. The payment stack — EMV, PIN entry, key management, the gateway, printing — is untouched certified code and stays exactly where it is. The redesigned screens are driven through the same interfaces the shipped screens are driven through, which means the process layer cannot tell which interface it is talking to.

This is worth restating for the risk register: the redesign changes what the merchant sees and touches. It does not change what happens to a card, a PIN, a key, or a message to the gateway.

3.1 The organising principle: one surface, no exits

Everything below follows from a single rule. If a merchant needs it to run a shop, it is inside BBPay. If a merchant does not need it, it does not exist on this terminal. There is no third category, and in particular there is no category of “available in Android if you know where to look”.

Applied honestly, that rule removes far more than it adds. Date and time, display timeout, developer options, app management, accounts, security, storage and about-phone are all deliberately absent. A terminal that offers those is a terminal that eventually gets broken by a curious cashier, and then gets reported to us as a payment fault.

3.2 The status bar — the app's own, not Android's

Android's system status bar is hidden. In its place sits the app's own strip, present on every screen, carrying exactly four things:

Element	Why it is there
The BBMSL / PCG lockup	Whose terminal this is. It is also the only branding surface a merchant sees all day, and it replaces a Google-styled bar with our own
Language button	The single highest-frequency setting on a bilingual counter, promoted from four levels inside Android Settings to one tap that is always on screen. It is labelled with the language you would switch to, not the one you are in — a merchant looking for Chinese looks for 中文, and a button showing the current language reads as a status rather than an action
Connection indicator	State, continuously, so that “is this thing online?” is answered before a customer is at the counter rather than after. The icon carries the state; the connection screen carries the detail
Battery, with percentage	A payment terminal that dies mid-shift is a closed till. A percentage, not a five-bar glyph, because the actionable question is “do I need to plug this in during lunch?”

That is the whole strip, and the restraint is deliberate. Every additional indicator is a thing a cashier must learn to ignore. We are open to feedback on the contents — merchants may tell us they want a fifth item, and the strip can carry one — but the starting position is that these four are what the counter needs and nothing else has earned a permanent place.

3.3 The payment loop

The flow is the shipped flow. It is stated here in full because its unchangedness is the load-bearing argument in Section 7.

Step	Screen	What the merchant does
1	Home	Starts a sale. Also the way to history, the connection screen, help and the manager surface
2	Amount	Types the amount on the same keypad, in the same arrangement, with the same confirm bar as today
3	Method	Chooses the payment method, rendered from the server's entitlements in the shipped display order
4	Present / scan	Card presented, or a QR code shown or scanned. Driven by the certified stack; the screen is presentation only
5	Result	Approved, declined, or unknown — with unknown kept as a first-class outcome that leads to reconciliation, exactly as the shipped app treats it
6	Back to Amount	Not back to home. The next sale starts where the last one ended

Step 6 is the only behavioural change in the loop and it is a subtraction, not an addition: it removes one tap per sale. A counter that never needs to check history or change a connection can stay on the keypad all day. A busy shop takes hundreds of payments and taps home hundreds of times for no reason; that tap is gone.

Cancel-and-retry is preserved as the shipped app implements it, including the fallback path where a failed contactless read re-enters the flow rather than dead-ending, and every transaction screen carries a Back control. These are not new conveniences. They are the shipped app's behaviours, which an earlier draft of the redesign lost and which were deliberately restored.

3.4 Two tiers, so that the cashier makes no decisions

Settings split along a single question: can getting this wrong cost money, or lose a connection, or change what the terminal is allowed to do? If yes, it is behind the manager passcode. If no, it is open.

Open to any cashier	Behind the manager passcode
Language (EN ⇄ 繁體中文)	Wi-Fi — scan, connect, forget
Text size (four steps)	Mobile data — enable, disable, roaming
Volume — beeps and prompts	Printer — copies, auto-print, test print, paper status
Connection status — read only	Account — username, change password
Help	Terminal — MID / TID, device ID, versions, auto-done timer

Open to any cashier	Behind the manager passcode
	Entitlements — read-only mirror of what the server permits
	Void, refund, settlement and the other manager transactions

Language and text size are open on purpose, and the reasoning is worth keeping: a cashier who cannot read the screen cannot call a manager to fix the fact that they cannot read the screen. Both settings change presentation only and carry no risk. Everything with a consequence is the manager's.

This is where the reduction in decision-making actually comes from. The manager configures the terminal once, for the shop they run and the staff they employ. From then on a cashier is not choosing between options — there are no options in front of them beyond the two that make the screen legible. The decision surface during a shift is the amount and the payment method, which is the decision surface the job actually has.

3.5 Pinned, and not by request

The application runs pinned. A merchant does not leave it to reach Android, because Android is not offered — there is no home button behaviour that exits, no recents, no notification shade. Servicing is a deliberate, authenticated act rather than a swipe.

The strength of that pin is the one thing in this report that depends on something we do not control, and Section 8 states the dependency rather than glossing it. The interface is identical either way; what changes is whether leaving the app is impossible or merely difficult.

4. Old against new, in detail

	Shipped app	Redesign
Mental model	“A payment app I open on a terminal”	“The terminal”
Top of screen	Android's status bar, then an ActionBar with a hamburger	One BBMSL strip: logo, language, connection, battery
Navigation	Full-width dark drawer with eleven transaction types, seven of them passcode-gated individually	Cashier home with the sale foremost; a manager surface behind one gate, showing only what this merchant is entitled to
Language	Android system locale. Not in the app	Status bar, one tap, instant, no restart
Wi-Fi / mobile data	Android Settings	In-app, manager tier
Text size	Android Settings → Display, which the app pins away in any case	Four in-app steps; amount and keypad exempt so the grid cannot break
Unavailable payment methods	Rendered dimmed rather than hidden, so a merchant can see what the terminal could accept	Preserved, unchanged — this was a good decision and we kept it
Actions the merchant is not entitled to	Shown in the drawer; pressing one produced a transaction the gateway refused	Not shown. An action nobody here can perform is not a disabled action, it is not an action
After a completed sale	Returns to home	Returns to the amount keypad
Unknown transaction outcome	First-class state; drops the merchant into history to reconcile	Preserved verbatim. It is a safety behaviour, not a loose end
Reachable from the terminal	The whole of Android	BBPay only
Certified payment code	—	Untouched

5. What this is worth

5.1 Fewer decisions at the counter, and therefore fewer errors

Every option visible to a cashier during a shift is either used or ignored, and the ignored ones are not free — they are visual load on a 5.5-inch screen and they are opportunities to press the wrong thing. The redesign presents a cashier with the sale, and with the two presentation controls that make the sale readable. Nothing else is on the surface, because the manager has already decided it.

5.2 A bilingual counter that actually works

Two staff members, two reading languages, one terminal, one second to switch, on any screen, without leaving the transaction. A shop can hire the staff it can find rather than the staff who can read the terminal. The translation asset already exists — 772 strings, translated for these merchants — and the redesign is largely the act of making it reachable.

Traditional Chinese specifically, not Simplified: the shipped locale is zh-Hant-TW and a Hong Kong merchant sees the difference immediately. The redesign resolves the locale explicitly rather than letting Android's default resolution turn 中文 into the wrong script.

5.3 Connectivity becomes a fact, not a discovery

The connection indicator is on screen during every sale, so an offline terminal is visible before a customer is at the counter. When it does need fixing, it is fixed in the app, at the manager tier, without an Android detour and — importantly — without the merchant being anywhere near the settings that are not about connectivity.

5.4 Support cost

A support call that begins “how do I change the language” or “how do I connect to the new Wi-Fi” is a call about Android, taken by our support team, at our cost, with our brand on the outcome. Those calls do not have an in-app answer today; after the redesign they have a one-sentence answer, and most of them will not be made at all. Section 9 proposes measuring exactly this.

5.5 A defensible boundary

This is the benefit that is easiest to underrate. Once the merchant experience is entirely inside our software, statements about how the terminal is used stop being hopes and become properties we can implement, observe and enforce. Concretely, a closed environment makes the following tractable in a way they simply are not on an open Android device:

- Attesting that the terminal is running our build, in the configuration we shipped, rather than a device that has been through an unknown sequence of settings changes.
- Binding a terminal to a location or a site, so that a unit which was deployed to one merchant and has physically moved elsewhere is visible to operations. The plumbing for this is straightforward once the app is the environment; on an open device it is trivially defeated.
- Enforcing that the terminal is used for what it was issued for — the app cannot be sidelined, replaced on screen, or used as a general-purpose tablet between sales.
- Making configuration drift impossible rather than merely discouraged, because there is no configuration surface outside the two tiers we designed.

- Recovering a terminal in the field to a known state, since the known state is the only state it has.

Location-linked and attestation features are enabled by this architecture, not delivered by it. They would be specified, scoped and — for anything touching location — assessed against the Personal Data (Privacy) Ordinance and disclosed to merchants in the terminal agreement before any of it is built. This report claims the capability becomes possible, not that it is done.

5.6 Fleet consistency

Every terminal in the field looks the same, behaves the same and can be talked through by a support agent reading from one script, because there is no per-device Android state to account for. A merchant with four terminals gets four identical terminals.

6. Design decisions worth defending individually

A few choices in the redesign will be questioned on their own merits rather than as part of the overall case. Each is deliberate.

6.1 The visual language is the shipped app's, not a new one

The redesign adopts the shipped palette rather than inventing one: the same teal primary, the same dark teal for the amount and keypad digits, the same surfaces, the same button and key radii, the same type scale. An earlier web prototype used a dark navy gradient aesthetic; it was abandoned precisely because a merchant who has used BBPay must recognise the redesign as BBPay on first sight. Familiarity is a feature we are buying deliberately, and it is the same purchase we make by keeping the flow.

6.2 Only two languages

English and Traditional Chinese, because that is what the shipped app is translated into and what this market reads. A language picker with a long list would be a worse product for the same merchants and a maintenance liability for strings nobody will proofread. If a market requires a third, the mechanism supports it; the shipping product does not pretend to.

6.3 The amount and the keypad do not scale with text size

They are already at the size limit of a 5.5-inch panel. Scaling them would wrap the total or break the keypad grid, and a wrapped total is worse than a small one. Body text, labels, list rows and buttons scale; the two elements that are already as large as the screen permits do not.

6.4 Entitlements filter the manager surface

A merchant who is not entitled to refunds does not see Refund. The shipped drawer showed all options unconditionally, which meant pressing one produced a transaction the gateway would refuse — a failure the merchant reasonably read as the terminal being broken. Showing only what this merchant may actually do removes a whole class of support call.

6.5 The result screen keeps “unknown”

Approved and declined are not the only outcomes. The shipped app treats an unknown gateway result as its own state, recovers it by querying the gateway rather than guessing, and drops the merchant into history to reconcile if the query also fails. That behaviour is preserved exactly. A redesign that modelled only success would be a redesign that occasionally loses money.

7. Objections, and our answers

The following are the arguments we expect, including the ones we consider strongest. Where an objection is partly correct we say so; a business case that concedes nothing is not a business case.

Objection 1 — Merchants will need a long time to adapt

The concern. Retraining hundreds of merchants and their staff is expensive, disruptive and risky. Payment terminals are muscle memory. A redesign resets that memory and the cost lands on us, in support calls and in merchant frustration, during the exact period when the product is least forgiving of mistakes.

Response. This is the right objection to raise and it is the one the design was built around. The answer is that the thing merchants have muscle memory for is not being changed. The payment sequence is identical: start a sale, type the amount on the same keypad in the same layout, choose a method from the same grid in the same order, present the card, read the result. The palette, the button shapes, the keypad geometry and the type scale are the shipped app's. A cashier who learns nothing at all can take payments on the first day.

What is new is almost entirely material that was not in the app before. Nobody has muscle memory for changing the language inside BBPay, because it was never possible inside BBPay. There is no habit to overwrite — only a task that used to require leaving the product and now does not. The redesign adds capability where there was none and preserves behaviour where there was some, which is the inverse of the profile that makes redesigns expensive.

The measurable form of this claim is in Section 9: if the flow is genuinely unchanged, time-to-first-sale on a redesigned terminal is within seconds of an existing one, and the pilot will show that before the fleet sees it.

Objection 2 — Touching a certified payment application is a certification risk

The concern. The shipped stack is certified. Any change to a certified product risks re-certification, which is slow and costly, and the risk is not worth a nicer interface.

Response. The redesign does not touch the certified boundary. EMV, PIN entry, key management, the gateway client and printing are unmodified. PIN entry in particular remains what it is today: the pad is drawn by the security processor from rectangles handed to it by the SDK, and app code never holds a PIN. The redesigned screens are driven through the same interfaces the shipped screens are driven through — the process layer emits the same events and waits on the same completions regardless of which set of screens is attached.

In other words the change is a presentation module inside the existing application, selected by build configuration, sitting above an unchanged domain layer. That is the smallest surface on which this product could be redesigned at all, and it was chosen for exactly this reason.

Objection 3 — Locking down Android removes flexibility we will wish we had

The concern. Today, if a merchant or an engineer needs something unforeseen — a certificate installed, a diagnostic run, a network quirk worked around — Android is there. Closing it forecloses solutions we have not thought of yet.

Response. Partly true, and it is the honest cost of the proposal. Our position is that the flexibility is real but is available to the wrong people. It is available to every cashier on every counter, all day, and it is used far more often by accident than by an engineer solving a problem.

The redesign does not remove servicing; it relocates it. Servicing becomes an authenticated, deliberate act rather than a swipe, and it belongs with the people who service terminals. The shipped app already contains the seed of this idea in its maintenance mode, which drops out of fullscreen for servicing — the redesign follows the same principle and makes it the only route.

It is also worth being concrete about what is being foreclosed. In three years of field support the recurring merchant-side uses of Android have been Wi-Fi, language, display and volume. All four are in the redesigned product, at the appropriate tier. The unforeseen-need argument is real in principle and thin in evidence.

Objection 4 — Managers will resent being restricted, or will want the full device

The concern. Merchants own their terminals in every practical sense. Telling a business owner what they may configure on their own counter is a customer-relations problem, and some managers will simply ask for the restriction to be lifted.

Response. The manager is not being restricted relative to the alternatives that matter; they are being given a control surface that did not exist. Today a manager has two options: leave the terminal's configuration to whoever happens to pick it up, or personally police an open Android device. The redesign gives them a third: set the terminal up once, behind their passcode, and know it will still be that way at closing.

The restriction is on the cashier, not on the manager, and it is a restriction managers ask for. What the manager loses is access to settings that no shop needs and that our support data shows are only ever changed by mistake — display timeout, developer options, app management, storage.

We would also note that the manager tier is not a fixed list handed down. It is the set of controls we believe a shop needs, and we are explicitly open to it growing. If merchants tell us they need something we did not include, that is feedback on the contents of a tier, not an argument against having tiers.

Objection 5 — Support and field engineering need Android access

The concern. Diagnostics, log collection, ADB, certificate installation and vendor tooling all live outside our app. A fully closed terminal makes the fleet harder for our own people to work on, not easier.

Response. The closed environment is closed to the merchant, not to the fleet management system. Terminal identifiers, application and SDK versions, printer and paper state, connection detail and entitlements are all surfaced in-app at the manager tier precisely so that a support call can be conducted without anyone leaving the product — which is faster than the current alternative of talking a shopkeeper through Android's About screen.

Beyond that, an authenticated maintenance route remains, and the vendor's own management channel is unaffected by anything in this proposal. If the fleet is placed under a device-management authority (Section 8), field engineering gains capability rather than losing it: remote configuration and remote recovery become properties of the fleet rather than site visits.

Objection 6 — The status bar is precious space, and a language button does not earn it

The concern. A 5.5-inch panel has very little vertical room. Spending a permanent strip on four items — one of which is a logo — costs screen area on every screen in the product, and language is a setting most merchants change once.

Response. Most merchants change it once. Bilingual counters change it several times a shift, and those are the merchants for whom the current product fails hardest. The cost of the button is a few millimetres; the cost of not having it is a support call and a cashier reading a screen in a language they parse slowly. Slow parsing at a payment terminal is how amounts get confirmed without being read.

The rest of the strip earns its place on the same test. Connection state prevents the most common bad moment in the product. Battery percentage prevents the second. The lockup is the only thing that tells a merchant, all day, whose terminal this is — and it is what stands where Android's status bar used to.

We hold this position loosely on contents and firmly on structure. Which four things sit in the strip is a question for merchant feedback. That the terminal has its own persistent strip rather than Android's is not.

Objection 7 — The lockdown can be defeated, so it is security theatre

The concern. If the app is pinned by an ordinary API call, a determined user leaves it by holding two buttons. Marketing a terminal as locked when it is not is worse than not claiming it at all.

Response. Correct, and the product does not claim otherwise. There are two versions of screen pinning: escapable, which any application may invoke, and locked, which requires the application to be Device Owner or to be allow-listed by one. On the current fleet only the first is available, and the settings screen reports the real state rather than implying a lockdown that is not there.

Even the escapable version does the larger part of the work, because the population it defends against is overwhelmingly accidental rather than adversarial: a cashier who tapped something and ended up outside the app. It does not defend against someone who wants out, and we do not say it does.

Closing that gap is a privilege decision, not an engineering one, and it is the single open item in Section 8. It is the reason this report asks for a decision rather than only an endorsement.

Objection 8 — Wi-Fi and mobile data cannot be controlled from an ordinary app anyway

The concern. Android 10 removed the ability for non-privileged applications to add or enable Wi-Fi networks, and toggling mobile data requires a privileged permission. An in-app connection screen therefore cannot do what it appears to offer.

Response. Technically accurate, and it is why the in-app connection screens were built against an abstract interface with a clearly-bounded set of operations — scan, connect, forget, set mobile data, report status. Three outcomes are possible depending on the privilege we hold, and the screens ship in all three.

Privilege held	Result
Device Owner, via the fleet management system	Full control. Device-owner applications retain the legacy network path. The loop stays closed
Vendor platform SDK with vendor signature	The terminal vendor's device-management interface exposes network control on their hardware. The loop stays closed
Neither	Fallback to the system's internet-connectivity panel — a slide-up sheet, not the whole of Android Settings. The loop bends; it does not break, and the merchant still never reaches Display or Developer options

The worst case is materially better than today, and only a small number of method bodies differ between the three. This objection changes the quality of one feature; it does not change whether the product ships.

Objection 9 — Older or less confident staff will struggle with a new interface

The concern. The merchant base includes staff who are not comfortable with technology. A redesign is precisely the population-level event that strands them.

Response. This population is the one the redesign helps most, and it is helped by the parts that are new rather than in spite of them. Text size is now a control they can reach without permission, in the product, in four steps. Language is one tap. Connection state is a picture on the screen instead of a diagnosis. Every one of those was previously locked behind an Android settings tree, which is exactly the surface that strands a less confident user.

Meanwhile the thing they already know — the sale — is unchanged. The redesign lowers the floor and does not move the part they are standing on.

Objection 10 — Location and attestation features raise privacy and legal questions

The concern. Binding a terminal to a location means processing location data. That carries obligations under the Personal Data (Privacy) Ordinance and may require merchant consent, disclosure and retention rules we have not written.

Response. Agreed, and this report deliberately does not treat those features as delivered. What the architecture provides is the precondition: a closed environment in which such a control could be implemented and would actually hold. Whether we implement one, what it records, how long it

is kept, and how it is disclosed to merchants are separate decisions requiring legal review and a change to the terminal agreement.

Nothing in the interface as it stands collects location. Section 5.5 claims a capability becomes possible; it does not claim it is switched on, and the sequencing keeps that distinction intact.

Objection 11 — This is a large engineering investment in something that already works

The concern. The terminal takes payments. Engineering time spent on the interface is time not spent on gateway features, new payment methods or acquiring integrations that generate revenue directly.

Response. The comparison is not interface work against revenue work; it is interface work against the recurring cost of not doing it. That cost is paid continuously in support calls that are Android tutoring, in terminals sitting in degraded configurations, in transactions delayed or abandoned at the counter, and in a merchant experience that visibly ends at the edge of our own screen.

It is also worth being clear about the scope. This is a presentation module above an unchanged domain layer, reusing the shipped palette, the shipped strings, the shipped payment sequence and the shipped process interfaces. The expensive parts of a terminal product — certification, the payment stack, the translation asset, the visual language — are all being reused rather than rebuilt.

And the investment compounds in a direction the current architecture cannot. Every subsequent thing we want the terminal to do — a new configuration control, a fleet policy, an attestation, a change in what a cashier may reach — is a change inside our own product once, rather than a negotiation with a general-purpose operating system on every device forever.

Objection 12 — Merchants did not ask for this

The concern. There is no merchant request on file for a redesigned interface. Merchants ask for lower rates and faster settlement, not for a new status bar.

Response. Merchants do not file requests for interface architecture; they file the symptoms of not having it. “How do I change the language.” “The terminal says it is not connected.” “My staff cannot read the screen.” “Someone changed something and now it looks different.” Those are the requests, and every one of them is a request for this.

The redesign is the general answer to a set of specific complaints that are currently answered one phone call at a time.

8. Dependencies and open items

One external dependency determines how completely the loop closes, and it is a commercial and administrative question rather than an engineering one. It is stated here in full rather than buried, because a business case that hides its dependency is not one that can be acted on.

Item	Question	Consequence if unresolved
Device Owner status	Can our package be granted Device Owner, or vendor-signed status, through the fleet management channel? On PCI-PTS terminals this privilege is normally held by the vendor or the acquirer rather than by us	Screen pinning remains escapable rather than locked, and in-app network control falls back to the system connectivity panel. The product still ships and is still a large improvement; the lockdown is a strong deterrent rather than an absolute
Network privileges	Follows from the above, or from access to the terminal vendor's device-management interface	Wi-Fi and mobile-data screens fall back as described in Objection 8
Location features	Legal review under the Personal Data (Privacy) Ordinance; merchant disclosure and terminal-agreement changes	Not built. The architecture supports it; nothing depends on it
Merchant feedback on the status bar	Do the four permanent indicators match what a counter needs?	Contents can change without affecting anything structural

The build order deliberately does not stall on any of these. The screens are implemented against interfaces with working fallbacks, so the answer to the privilege question changes a small number of method bodies rather than the shape of the product.

9. Rollout and how we will know it worked

9.1 Phasing

1. Internal validation on physical P5 hardware — every screen, in both languages, at the largest text size, which is the worst-case layout and the one that breaks first.
2. Pilot with a small set of merchants chosen to include at least one bilingual counter, one multi-terminal site and one merchant with staff who are known to find the current terminal difficult.
3. Measurement window against the baseline in 9.2, long enough to include a month-end settlement.
4. Staged fleet rollout by cohort, with the previous interface available as a build-level fallback until the pilot metrics are confirmed at scale.

9.2 Metrics

The adoption objection is the one this report leans on hardest, so it is also the one we propose to measure most directly.

Measure	Why it matters	Target
Time from terminal idle to completed first sale, new staff, unassisted	Directly tests the “long adoption time” objection	Within seconds of the current interface, from day one
Support contacts categorised as configuration or Android navigation	The cost the redesign is intended to remove	Substantial reduction against the pre-pilot baseline
Language switches per terminal per day	Evidence for whether the promoted control was the right call	Non-trivial usage on bilingual counters
Sales abandoned or retried after a connectivity failure	Tests whether persistent connection state changes behaviour	Reduction against baseline
Terminals found in a non-standard configuration during support	Tests the closed-environment claim	Approaching zero
Merchant-reported difficulty during the first week	Catches the objection this report may have got wrong	No material increase

A baseline for each of these should be captured on the current fleet before the pilot starts. Without it the comparison is anecdote, and the adoption argument in Section 7 deserves better than anecdote.

10. Recommendation

Adopt the redesigned interface as the shipping merchant surface for BBPay on the P5 fleet, and proceed with the pilot described in Section 9.

The case rests on three things. First, the product currently sends merchants out of itself to do four of the seven things they most often need, and that is the whole of the problem being solved. Second, the redesign solves it without touching a line of certified payment code and without changing the payment sequence a merchant already performs — which is why the adoption cost that would normally sink a terminal redesign does not apply here. Third, closing the environment turns a set of properties we currently hope for into properties we can implement and enforce.

Alongside adoption, we ask for a decision on the Device Owner question in Section 8. The interface ships either way, but that decision is the difference between a terminal that is difficult to leave and one that cannot be left, and it also determines whether connection management stays entirely inside our product.

Feedback on the contents of the status bar, on the split between the open and manager tiers, and on anything a shop needs that we have not included is actively wanted. The architecture is a position we hold firmly; the inventory inside it is not, and it is easier to change now than after the fleet has it.